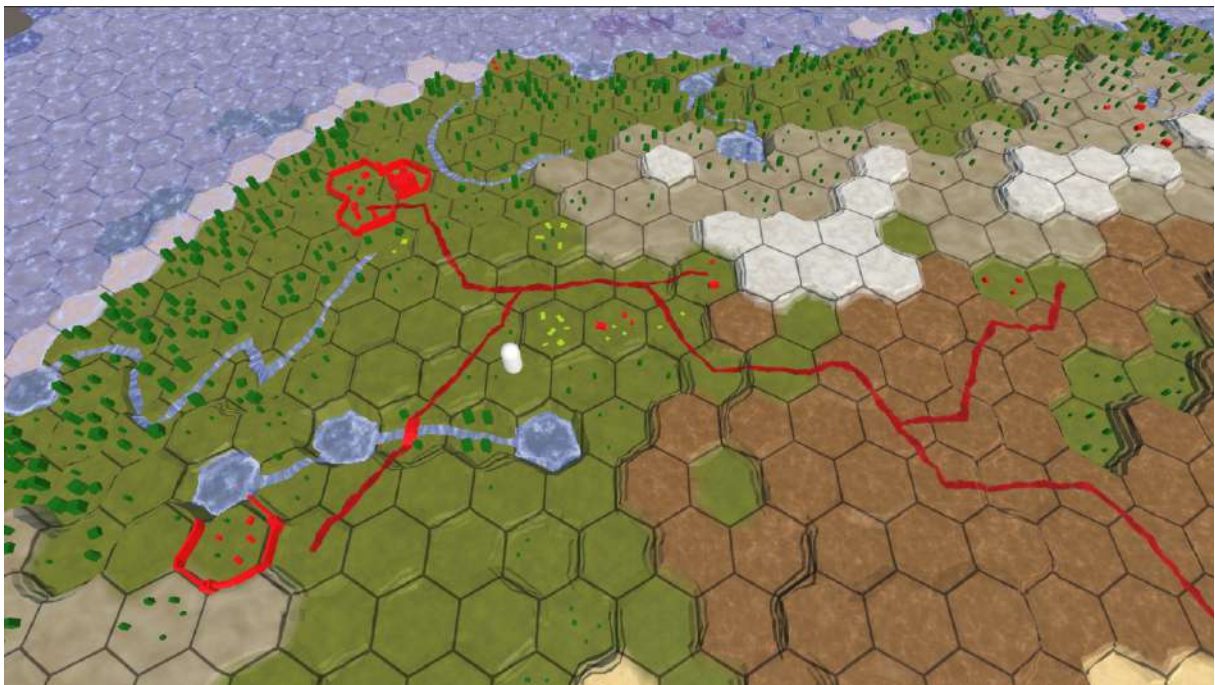


Dossier créatif master JMIN 2022

Path Builder

Auteur : Moisan Simon

N° candidat : CAN2097



Mots clés : Jeu d'exploration et de stratégie, 3D, Génération Procédurale, Expérience de jeu

Sujet du dossier créatif

"En marchant se construit le chemin" d'Antonio MACHADO (1875-1939)

*Marcheur, ce sont tes traces
Ce chemin, et rien de plus ;
Marcheur, il n'y a pas de chemin,
Le chemin se construit en marchant.
En marchant se construit le chemin,
Et en regardant en arrière
On voit la sente que jamais
On ne foulera à nouveau.
Marcheur, il n'y a pas de chemin,
Seulement des sillages sur la mer.*

Sommaire

I. Partie commune	3
I.1. Concept du jeu	3
I.2. L'expérience de jeu	3
I.3. La direction artistique	4
I.4. La direction sonore	4
I.5. Problématique technique	4
I.6. Le public cible	4
II. Partie spécialité programmation	5
II.1. Problématique	5
II.2. Bibliographie	5
II.2.1 La génération procédurale au service de l'expérience de jeu	5
II.2.2 Une architecture de génération procédurale	6
II.3. Les solutions	7
Solution 1 : Base de données de jeu	7
Solution 2 : Modèles de génération	8
Conclusion	8

I. Partie commune

I.1. Concept du jeu

À une époque où la civilisation humaine connaît encore l'âge d'or des **grandes découvertes**, un gigantesque archipel inconnu et inhabité a été découvert. Un jeune agent de la **guilde des architectes**, incarné par le joueur, a été envoyé sur l'une d'entre elles pour participer à son développement en supervisant la construction de **voies**. Pour cela il devra d'abord explorer les contrées sauvages de l'île, récolter des ressources et rencontrer les habitants qui s'y sont installés. Ensuite, le joueur pourra les aider à se développer en les reliant entre eux avec divers types de voies.

I.2. L'expérience de jeu

Path Builder est un jeu de **stratégie** et d'**exploration** en **3D** vue du dessus. Il propose une expérience offrant le plus de **liberté** possible au joueur pour réaliser l'objectif du jeu. Ce dernier, qui est de participer à la **prospérité** des nouveaux habitants d'une île, est volontairement laissé **ouvert**. En effet, le joueur peut vivre sa propre **aventure** et décider comment il veut remplir sa mission. Cela est possible grâce à une boucle de gameplay principal simple et accessible. Elle repose d'abord sur l'exploration d'une île générée de manière procédurale, dont les tuiles hexagonales sont masquées par un brouillard de guerre. Chaque tuile

contient des éléments qui peuvent être bénéfiques au joueur ou bien représenter un obstacle pour ses déplacements ou pour ses constructions. Ces mécaniques d'exploration sont inspirées des jeux **Civilization** [1] et **Curious Expedition** [2].

Ensuite, le joueur pourra collecter diverses **ressources** dans la nature avec ses **outils**. Aussi, en découvrant des



habitations il pourra interagir avec des **pnjs**, pour discuter avec eux, faire des échanges, réaliser des **quêtes** et aussi engager de la **main-d'oeuvre**. Ce modèle d'interaction est inspiré du jeu **Stardew Valley** [10].

Avec ses nouvelles connaissances du terrain et des **besoins** des habitants rencontrés, le joueur va pouvoir **choisir** à qui ses voies vont profiter et de quelle manière. Le déroulement du jeu fonctionne autour d'une boucle de feedbacks positifs initiée et entretenue par le joueur. Les voies construites au long de la partie vont permettre aux habitations et à leurs activités d'évoluer, d'échanger entre eux et d'avoir accès aux ressources de l'île. Cette progression va permettre au joueur de débloquent des améliorations pour faire évoluer la boucle de gameplay. Pour finir, le joueur pourra décider quand changer d'île pour découvrir des variations d'environnement et de gameplay.



I.3. La direction artistique

En ce qui concerne l'aspect visuel, la caméra en vue du dessus plongé permet d'avoir une vue d'ensemble des parties explorées ainsi que de la topologie de l'île. Les éléments 3Ds du jeu sont modélisés en **low-poly**, à la manière du jeu **For The King** [3]. Ce style épuré est facile à créer, il permet de faciliter la **lisibilité** et de favoriser les **performances** d'affichage.

I.4. La direction sonore

Le sound design de Path Builder se caractérise par une bande sonore avec un tempo assez lent pour favoriser une ambiance **calme** et **posée**. Ensuite, la bande sonde intègre un **dynamisme continu** en se **rajoutant/retirant** des instruments ou des effets sonores en fonction de ce qui entoure le joueur. Par exemple, pendant l'exploration des contrées sauvages on peut intégrer des bruitages de la faune et de la flore, ou en étant à proximité d'une ferme on peut ajouter des bruitages évoquant l'activité agricole. On retrouve ce dynamisme dans les jeux **Left4Dead 2** [6] et **Outer Wilds** [8].

I.5. Problématique technique

L'aspect technique de Path Builder que j'ai choisi est la **génération procédurale** de la **topologie** des îles de l'archipel. Pour cela, on peut utiliser une succession de **noise map de Perlin** avec différentes fréquences et amplitudes. Ensuite, on peut obtenir la forme d'une île en soustrayant un **gradient carré** à la noise map pour atténuer les valeurs en bordure de carte. Enfin, chaque valeur de la map va correspondre à la **hauteur** des tuiles de l'île. Cette méthode est inspirée des vidéos sur la génération procédurale de continents par **Sebastian Lague** [14].

I.6. Le public cible

Path Builder se veut le plus **accessible** possible et est jouable sur **PC**, **consoles** et sur **mobile**. Le public cible est des joueurs de **15-30 ans** intéressés par des jeux à l'**ambiance calme, facile d'accès** et **personnalisable**. Enfin, Path Builder met à disposition des options pour modifier certaines **règles** et **paramètres** de génération de l'île. On retrouve aussi ce genre d'options dans le jeu **Don't Starve** [4].

II. Partie spécialité

II.1. Problématique

La **génération procédurale** est une technique utilisée dans les jeux vidéo qui consiste à utiliser des algorithmes pour créer du contenu que ce soit des objets ou des niveaux entiers. Cette pratique a même donné naissance à un genre de jeu à part entière : le **Rogue-like**, en hommage au jeu du même nom [9].

La Génération Procédurale de Contenu (**GPC**) [5] qui s’oppose au contenu fait main, peut s’avérer très bénéfique. En effet, elle permet de **réduire le temps** de production de contenu en permettant au jeu de créer ses niveaux.

Un aspect que je trouve intéressant avec l’utilisation de la GPC pour créer des niveaux, c’est ce que cela apporte à l’**expérience** de jeu. Cette technique permet au joueur de répéter la boucle de gameplay, d’apprendre et d’assimiler les mécaniques du jeu, le tout en ayant une sensation de fraîcheur à chaque partie. Mais pour cela, il faut que les niveaux générés soient cohérents et suffisamment variés.

C’est pourquoi je me suis penché sur cet aspect en proposant des solutions à la problématique suivante : **Comment utiliser la génération procédurale pour créer des niveaux à la fois cohérents et variés ?**

Aussi j’ai pris la génération procédurale des îles de l’archipel du jeu **Path Builder** comme

contexte de mise en place des solutions.

II.2. Bibliographie

II.2.1 La génération procédurale au service de l’expérience de jeu

La conférence **”Procedural Worlds, Handmade Details”** de Mark Johnson [12] réalisée pendant la **GDC 2015** traite des avantages de la génération procédurale sur l’expérience de jeu. En effet, l’aspect pseudo-aléatoire de la génération procédurale apporte de la **variété**, de la **rejouabilité** et des situations **imprévisibles** et **uniques** au jeu. Mais cette conférence aborde aussi le sujet de l’**équilibre** entre le contenu généré **procéduralement** et le contenu **fait main** dans un jeu vidéo. Plusieurs exemples de jeux y sont présentés ainsi que la manière dont ces deux méthodes de création sont **imbriquées**. Un générateur procédural consiste à combiner des éléments de jeu fait main (aussi appelés **préfabriqués**) entre eux, pour créer un ensemble **unique**. Ceci dit, une grande variété de niveau n’est pas toujours synonyme de **qualité**. En effet, même si les niveaux sont tous différents, ils peuvent être très similaires dans la manière dont ils sont générés. Aussi, l’aspect aléatoire de la génération devient rapidement visible si les composants du niveau n’ont aucun lien entre eux.

Par exemple, dans le jeu d’exploration spatiale **No Man’s Sky** [7], toutes les planètes de l’univers de ce jeu sont générées

procéduralement. Bien que cela représente **18 quintillions de planètes** potentielles à visiter, de nombreux joueurs ont été déçu par leur **redondance** et surtout par leur manque d'**identité**.

C'est pourquoi dans certains jeux des **règles** sont mis en place pour **interconnecter** les contenus générés entre eux. Cela permet d'avoir une **continuité** dans la génération, mais aussi de la **cohérence**. C'est le cas par exemple du Rogue-Like **Faster Than Light** qui intègre des quêtes dans la génération procédurale de ses secteurs. Ces quêtes, qui sont préfabriquées, nécessitent que le joueur passe par au moins deux balises différentes, la première pour commencer la quête et la deuxième pour la conclure. Pour que les quêtes soient **finissables**, le jeu prend soin de placer les balises à visiter dans la direction opposée à la flotte rebelle que le joueur doit fuir, quitte à les placer dans le secteur suivant.

II.2.2 Une architecture de génération procédurale

Ultima Ratio Regum [13] est un jeu vidéo créé par **Mark R Johnson** qui se déroule dans un monde vaste et complexe entièrement généré de manière procédurale. Le but de chaque partie générée par le jeu est d'enquêter sur une **conspiration** en interagissant avec diverses **nations**. Ces dernières sont assez **complexes** car elles possèdent leurs propres villes, cultures, religions, relations diploma-

tiques, blason, design de mobilier etc.. Mais ce qui fait la force de ce jeu, c'est que tous ces types de contenus différents sont générés de sorte à être **connecté** les uns avec les autres. En effet, la génération est divisée en **modèles** qui sont exécutés dans un **ordre** spécifique et reliés entre eux par des **chaînes logiques**. Chaque modèle va générer un type de contenu particulier avec ses propres règles et le résultat en sortie aura une **influence** sur les modèles suivants. Par exemple, le symbole du blason d'une nation peut avoir un lien avec un animal considéré comme sacré dans leur religion.

Ensuite, ce jeu intègre un système de **probabilité pondéré** dans ses règles de génération. Autrement dit, les probabilités de génération des différentes versions d'un contenu ne sont **pas équitables**. Il est possible de tomber sur des versions générées **communes**, mais il est aussi possible de tomber plus rarement sur des versions **uniques fait main**. Ces différentes versions possèdent un **poids** qui est lui aussi impacté par les chaînes logiques des modèles de sorte à éviter les **redondances** au sein d'une partie. Le but est de créer une **identité propre** à chaque nations et ainsi éviter le sentiment de **déjà-vu**. L'utilisation d'une telle architecture de génération procédurale profite grandement à l'expérience de jeu car elle permet de générer des environnements complexes, cohérents et suffisamment variés pour donner envie au joueur de s'y plonger.

II.3. Les solutions techniques

Solution 1 : Base de données de jeu

La première solution que j'ai à proposer est un **système de récupération et de caractérisation des données de jeu**. Cela va permettre de garder une **trace** des contenus rencontrés par le joueur et **adapter** la génération des prochains niveaux en conséquence. Trois informations vont nous intéresser :

- **Les labels** : ils vont caractériser les versions possibles d'un contenu.
- Le **nombre d'apparition** des labels.
- Le **poids** : c'est une valeur qui va influencer la probabilité d'apparition de contenu correspondant à son label.

Dans **Path Builder** par exemple, on peut caractériser le **climat** d'une île avec des labels tel que "arctique", "désertique", "tempéré" ou "tropical", en se basant sur le **biome** majoritairement présent sur les tuiles, la **température** et le taux d'humidité. Les poids des labels quant à eux possèdent une **valeur de base** correspondant à leur chance initiale d'apparition. Cette valeur est de 10 pour les labels dit "**commun**", mais elle peut être inférieur pour des labels **uniques**. Mais ce qui compte, c'est la **valeur finale** des poids qui est calculée en fonction du nombre d'apparitions des labels. On peut tout à fait envisager une formule comme celle-ci :

$$PoidsFinal = \frac{PoidsBase}{1 + NombreApparition}$$

Avec ces informations, on va pouvoir **influencer** la génération pour que le joueur ait plus de chances de tomber sur des contenus ou des combinaisons de contenu qu'il n'a pas encore ou peu vu.

Une manière de faire est de passer les poids en **paramètre** du générateur pour qu'ils impactent l'aléatoire des paramètres de génération. Mais cela peut être assez compliqué à mettre en place. C'est pourquoi il est aussi possible de générer **plusieurs îles**, chacune avec un poids égal à la somme des poids de leurs labels. On sélectionne ensuite une île au hasard et l'île avec le poids le plus grand aura plus de **chances** d'être sélectionné.

L'utilisation de **ScriptableObjects** (SO) du moteur de jeu **Unity** serait très utile pour stocker les données. On peut facilement y intégrer des **structures de stockage** et ils peuvent aussi contenir des **fonctions** qui pourront être appelé depuis les scripts de gameplay.

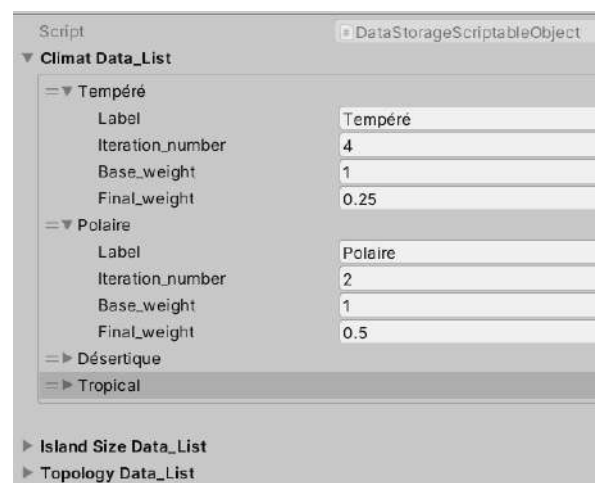
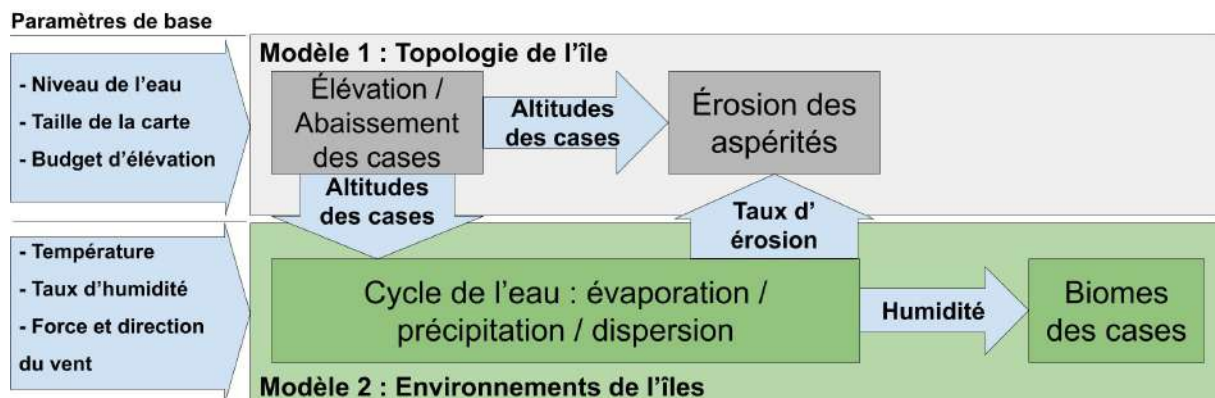


Image 1 : Capture d'écran d'un prototype de ScriptableObject



Solution 2 : Modèles de génération

Ma deuxième solution est une **architecture** pour la génération procédurale de niveaux. Elle est composée d'une succession de **modèles**, un pour chaque couche de contenu du niveau. Tous ces modèles sont articulés les uns avec les autres, à la manière du jeu **Ultima Ratio Regum** vu précédemment.

Pour illustrer le fonctionnement de cette solution, j'ai expérimenté sur un projet Unity de génération procédurale de terrain créé par **Jasper Flick** [11]. J'ai tout d'abord réglé les paramètres par défaut pour que le terrain généré soit une île pour être raccord avec le jeu Path Builder. Ensuite, je me suis penché sur la création de **deux modèles** qui regroupent plusieurs algorithmes de génération :

- **Modèle de topologie** : il va définir la forme de l'île en élevant/abaissant les tuiles.
- **Modèle des environnements** : il va définir le biome de chaque case de la carte

Le fait que les deux modèles aient besoin l'un de l'autre pour générer leur contenu permet de créer de la **cohérence** entre ces deux aspects

de l'île. Ensuite, le fait que les paramètres de base soient aléatoires permet de créer des combinaisons **variées** de contenu. Il est aussi possible d'altérer cet aspect en intégrant des **paramètres extérieurs**, tel que les poids de la **Solution 1**, donnant ainsi plus ou moins de chance à un type de contenu d'être généré.

Ce type d'architecture est très **modulable**, la chaîne peut être étendue en ajoutant et en interconnectant de nouveaux modèles. Le fait d'utiliser Cette architecture profite à l'expérience de jeu de par la qualité du contenu généré, mais elle peut prendre plus ou moins de temps à être mis en place en fonction du nombre de modèles.

Conclusion

Ces deux solutions permettent de créer une continuité entre les contenus d'une partie, mais aussi entre les parties elles-mêmes. L'idée derrière ça est d'avoir une progression unique et cohérente de l'expérience de jeu. Il est possible d'étendre cette idée en ajoutant du nouveau contenu à débloquer au fil du jeu.

References

- [1] Civilization v — game. <https://civilization.com/fr-FR/civilization-5>, 2022. Accessed 23 January 2022.
- [2] Curious expdition — game. <https://curious-expedition.com/index2.html>, 2022. Accessed 23 January 2022.
- [3] Curve games — for the king. <https://curvegames.com/our-games/for-the-king/>, 2022. Accessed 23 January 2022.
- [4] Don't starve - klei. <https://www.klei.com/games/dont-starve>, 2022. Accessed 23 January 2022.
- [5] Génération procédurale - wiki. https://fr.wikipedia.org/wiki/G%C3%A9n%C3%A9ration_proc%C3%A9durale, 2022. Accessed 23 January 2022.
- [6] Left 4 dead 2 — game. <https://www.l4d.com/game.html>, 2022. Accessed 23 January 2022.
- [7] No man's sky - game. <https://www.nomanssky.com/>, 2022. Accessed 23 January 2022.
- [8] Outer wilds — game. <https://www.mobiusdigitalgames.com/outer-wilds.html>, 2022. Accessed 23 January 2022.
- [9] Rogue - game. [https://fr.wikipedia.org/wiki/Rogue_\(jeu_vid%C3%A9o\)](https://fr.wikipedia.org/wiki/Rogue_(jeu_vid%C3%A9o)), 2022. Accessed 23 January 2022.
- [10] Stardew valley. <https://www.stardewvalley.net/>, 2022. Accessed 23 January 2022.
- [11] Jasper Flick. Hex map - tutorial. <https://catlikecoding.com/unity/tutorials/hex-map/>, 2022. Accessed 23 January 2022.
- [12] Mark R Johnson. Procedural worlds, handmade details - video. https://www.youtube.com/watch?v=Z_1T6f2ThzI&t=8s, 2022. Accessed 23 January 2022.
- [13] Mark R Johnson. Ultima ratio regum - game. <https://www.markrjohnsongames.com/games/ultima-ratio-regum/>, 2022. Accessed 23 January 2022.

- [14] Sebastian Lague. Landmass generator tutorial series. https://www.youtube.com/watch?v=wbpMiKiSKm8&list=PLFt_AvWsXl0eBW2EiBtl_sxmDtSgZBxB3, 2022. Accessed 23 January 2022.